DESHMUKH TECHNOLOGIES

DTTP · V1.1

# Self-Growth Program

## DTTP Full-Stack Developer Self-Growth Roadmap

Ten learning levels that turn full-stack reference material into a practical Deshmukh Technologies growth path — from web fundamentals to professional engineering.

| | |
|---|---|
| PURPOSE | LEVELS |
| Self-directed full-stack growth | 10 progressive stages |
| TRACKS | FRONTEND |
| Java / Spring Boot or Python / FastAPI | React + TypeScript |
| PRACTICE STACK | COMPANION |
| SQL + GitHub + Docker + CI/CD + AWS | 90-Day DTTP Handbook |

*Learn. Build. Solve. Review. Deploy. Grow.*

INTERNAL TALENT DEVELOPMENT STANDARD

| DOCUMENT | VERSION | CLASSIFICATION | OWNER |
| --- | --- | --- | --- |
| DTTP Self-Growth Program | 1.1 | Internal Training Standard | Deshmukh Technologies |

ROADMAP

# Contents

## 00 How to use this program

Full-stack reference material provides a strong foundation. It is **not** the entire DTTP curriculum. Use it as the **Full-Stack Knowledge Reference**, then apply it through Deshmukh Technologies tracks and real projects.

> Do not ask trainees to study the reference as one large syllabus. Complete one level before treating the next as the main focus. Combine each level with Java/Spring Boot or Python/FastAPI, React/TypeScript, SQL, GitHub, Docker, CI/CD, AWS, and real Deshmukh Technologies work.

| Document | Role |
|---|---|
| 90-Day DTTP Handbook | Timed training path, project, assessment, and graduation |
| Self-Growth Program | Deeper knowledge map for study before, during, and after the 90 days |
| Full-Stack Reference | Topic encyclopedia — not a replacement for practice |

**STUDY METHOD**

1. Read the concept.
2. Explain it in your own words.
3. Build a small example.
4. Break it, then debug it.
5. Commit the work.
6. Ask a mentor to review it.

**WEEKLY HABIT**

- 20% theory
- 30% exercises
- 40% project work
- 10% review and notes

01 **Web Fundamentals**

02 **Frontend Fundamentals**

03 **React + TypeScript**

04 **HTTP + REST + Web APIs**

05 **Java / Python Backend**

06 **Databases**

07 **Architecture**

08 **Testing + Security + Performance**

09 **Git + Docker + CI/CD + Cloud**

10 **Agile + Professional Engineering**

The destination is a **self-directed full-stack developer** who can build, test, debug, explain, document, and maintain software.

## 01 Level 1 — Web Fundamentals

**Purpose:** give the trainee a mental model of the web before frameworks.

### STUDY

- Client and server roles
- Browser, DNS, domain, IP address, port
- URL: protocol, host, path, query, fragment
- HTTP as request/response
- Static page vs dynamic application
- HTML structure, CSS presentation, JavaScript behavior
- Server-side logic and the meaning of full-stack
- Editor, DevTools, terminal, package manager, Git

### MUST BE ABLE TO EXPLAIN

- What happens after a user types a URL
- Difference between frontend and backend
- File server vs application server
- Why JavaScript runs in the browser and Java/Python run on the server

### PRACTICE

- Draw browser → DNS → server → response
- Build `index.html`, `styles.css`, `app.js`
- Inspect Elements, Network, and Console
- Serve a static folder locally

- [ ] Can explain client, server, URL, DNS, and HTTP
- [ ] Can build and style a simple page
- [ ] Can inspect HTML, CSS, and network calls
- [ ] Can describe what a full-stack application contains

> A trainee who cannot explain how a browser request reaches a server is not ready for React or Spring Boot.

## 02 Level 2 — Frontend Engineering

**Purpose:** make the trainee fluent in the browser before React abstractions.

### STRUCTURE & STYLE

- Semantic HTML, landmarks, tables, lists
- Forms, labels, input types, native validation
- Cascade, selectors, specificity, inheritance
- Box model, Flexbox, CSS Grid
- Responsive units and media queries

### BEHAVIOR & ACCESS

- Variables, types, scope, functions
- Objects, arrays, errors, `try / catch`
- DOM events, create/update/remove nodes
- Fetch: JSON, loading, error states
- Storage, timers, history
- Accessibility: keyboard, contrast, labels, focus

### PRACTICE

Responsive landing page | Validated form | DOM todo list | Public JSON API render | Keyboard-only use

- [ ] Can layout a page with Flexbox and Grid
- [ ] Can handle form input and validation
- [ ] Can update the DOM from JavaScript
- [ ] Can fetch JSON and render success/error/loading

- [ ] Can name at least three accessibility checks

## 03 Level 3 — React & Modern Frontend

**Purpose:** build maintainable UI with **React + TypeScript**.

Components | Props vs state | Effects / lifecycle | Controlled forms | Composition | Context API | Routing | Protected routes | SPA structure | Typed API responses

```
src/
├── components/   pages/   layouts/   services/
├── hooks/        types/   utils/     routes/
└── assets/
```

Practice: multi-page app, login + protected route, form POST, loading/empty/error states, reusable table. Apply this to the Employee Management System frontend.

- [ ] Can create typed React components
- [ ] Can manage local and shared state
- [ ] Can implement routing and a protected page
- [ ] Can integrate an API through a service layer
- [ ] Can keep UI responsive and readable

## 04 Level 4 — Web Communication & APIs

**Purpose:** teach what actually happens between frontend and backend.

### HTTP

- Request/response line, headers, body
- GET, POST, PUT, PATCH, DELETE
- Safe methods and idempotency
- 200, 201, 204, 400, 401, 403, 404, 409, 422, 500
- Cookies, Content-Type, CORS preflight

### Realtime

- Short and long polling
- Server-Sent Events
- WebSockets
- When each model is the right choice

### API styles

- JSON as a data contract
- REST resources and error payloads
- Pagination and filtering
- GraphQL as a query API
- OpenAPI / Swagger

Both backend tracks implement the same Employee Management API so Java and Python trainees learn the same design.

```
POST /api/auth/login
GET|POST /api/employees    GET|PUT|DELETE /api/employees/{id}
GET|POST /api/departments  GET|POST /api/attendance
GET|POST /api/leaves       PUT /api/leaves/{id}/approve
```

- [ ] Can read a network trace and explain each part
- [ ] Can choose the correct HTTP method and status code
- [ ] Can describe REST resource design
- [ ] Can say when polling, SSE, or WebSockets is appropriate

## 05 Level 5 — Backend Development

**Purpose:** implement server-side business logic on one DTTP track. Trainees are not required to learn every backend language in the reference.

### SHARED BACKEND SKILLS

Validation | DTO / models | Service layer | Repository | Errors & logging | Authn / authz | Environment config

### JAVA TRACK

JAVA | SPRING BOOT | REST | JPA / HIBERNATE | SPRING SECURITY | JUNIT / MOCKITO

Controller → Service → Repository → JPA / Hibernate → Database

### PYTHON TRACK

PYTHON | FASTAPI | PYDANTIC | SQLALCHEMY | AUTHENTICATION | PYTEST

Router → Service → Repository → SQLAlchemy → Database

Practice: login, employee CRUD, departments, attendance, leave apply/approve, consistent error JSON, one unit test per service.

- [ ] Can implement a validated REST endpoint
- [ ] Can separate controller, service, and repository
- [ ] Can persist and read data through the ORM
- [ ] Can protect at least one endpoint with authentication
- [ ] Can explain a request through every backend layer

## 06 Level 6 — Database Engineering

**Purpose:** make SQL a first-class skill. **SQL + MySQL / PostgreSQL is mandatory.** NoSQL is introduced later.

**RELATIONAL — MANDATORY**

- Tables, types, keys, constraints
- One-to-one, one-to-many, many-to-many
- SELECT, INSERT, UPDATE, DELETE
- JOIN, GROUP BY, HAVING, indexes
- Transactions and isolation
- Normalization and query plans
- ORM mapping and N+1 awareness

**NON-RELATIONAL — LATER**

- Key-value stores
- Document databases
- Graph databases
- Column-oriented databases
- When SQL is still the better default

Practice: schema for employees, departments, attendance, leave; join reports; justified index; all-or-nothing transaction; same query in SQL and ORM.

- [ ] Can design a normalized schema for the training project
- [ ] Can write joins and aggregations by hand
- [ ] Can use transactions correctly
- [ ] Can explain an ORM query in SQL
- [ ] Can name one valid later use of NoSQL

> ORM knowledge must not replace SQL knowledge.

## 07 Level 7 — Architecture

**Purpose:** move from developer thinking to engineer thinking. Study models, but understand **why** architecture changes.

Client-server | Layered / n-tier | Monolith | Modular monolith | SOA | Microservices | Component-based | Microfrontends | Messaging | MVC | MVP | MVVM | Coupling / cohesion

> Do not teach microservices first. Teach Monolith → Layered Architecture → Modular Monolith → Distributed Systems → Microservices.

Practice: draw Employee Management as a layered monolith; identify auth, employees, attendance, leave, reports; list three reasons not to split them into services yet.

Both tracks use the same shape: **React → REST → Service → Persistence → MySQL / PostgreSQL**.

- [ ] Can draw the current training system
- [ ] Can name each layer and its responsibility
- [ ] Can explain monolith vs microservices without slogans
- [ ] Can justify the DTTP architecture sequence

## 08 Level 8 — Testing, Security & Performance

**Purpose:** make quality a permanent habit, not a late phase.

### Testing

- Unit tests for business rules
- Integration tests for API and database
- TDD as discipline, coverage as signal
- Doubles: dummy, stub, spy, mock, fake
- Frontend component, form, and error-state tests
- Postman collections for the business API

### Security

- SQL injection and XSS
- Authentication vs authorization
- Password hashing, JWT risks
- Roles, secrets, environment variables
- TLS / HTTPS, CORS, CSP
- Least privilege and security headers

### Performance

- Measure before optimizing
- TTFB, payload size, query time
- Server- and client-side caching
- Images, minification, compression
- Lazy loading and preloading
- Slow SQL and N+1 queries

- [ ] Can write unit and API tests
- [ ] Can explain authn vs authz
- [ ] Can keep secrets out of source control
- [ ] Can measure one performance number and improve it
- [ ] Can name the main project security risks

> Never commit passwords, API keys, tokens, private keys, or production credentials to GitHub. Measure performance before optimizing it.

## 09 Level 9 — DevOps & Deployment

**Purpose:** connect code to a running, repeatable delivery path.

### Git

- Working tree, staging, commit
- Meaningful messages
- Branch per task, PR, review
- Merge, rebase, conflicts
- Revert and stash

### Docker

- Image vs container
- Dockerfile
- Volumes, networks, env vars
- Compose: frontend, backend, database

### Deployment

- Build artifacts and hosting
- Container deployment
- Health checks and logs
- Rollback thinking

**DTTP DELIVERY PATH**

GIT | GITHUB | GITHUB ACTIONS | DOCKER | AWS

Pipeline: Developer → GitHub → Build → Lint → Unit Tests → Integration Tests → Docker Build → Artifact → Deployment → Health Check.

AWS introduction: IAM, EC2, S3, RDS, CloudFront, Route 53, Security Groups, CloudWatch. Understand the purpose of each service before using it.

- [ ] Can complete a branch / PR / review / merge cycle
- [ ] Can run the project with Docker Compose
- [ ] Can describe a CI pipeline
- [ ] Can explain the purpose of the core AWS services
- [ ] Can diagnose a failed deploy from logs

## 10 Level 10 — Professional Software Engineering

**Purpose:** move beyond technology into how professional teams work.

### AGILE & SCRUM

- Agile principles
- Roles: Product Owner, Scrum Master, Developers
- Events: planning, daily, review, retrospective
- Artifacts: product backlog, sprint backlog, increment
- User stories and acceptance criteria

### PROFESSIONAL SKILLS

- Stand-up and written communication
- Requirement analysis and documentation
- Code review: clarity, correctness, security, tests
- Estimation, ownership, asking for help early
- Technical presentation and peer mentoring

| Level | Example |
|---|---|
| Epic | Employee Management |
| Feature | Leave |
| User story | As an HR user, I want to approve leave so that attendance stays accurate. |
| Task | Create leave approve API |
| Subtasks | Entity, DTO, service rule, controller, test, PR |

Daily report: objective, completed work, learning, problems, investigation, resolution, Git activity, next plan.

- [ ] Can write a user story with acceptance criteria
- [ ] Can break work into tasks and subtasks
- [ ] Can review code with specific comments
- [ ] Can present a feature to a mentor
- [ ] Can describe their own skill gaps honestly

## 11 Official progression

| Level | Focus | Outcome |
|---|---|---|
| 1 | Web Fundamentals | Can explain how a browser request becomes a response |
| 2 | Frontend Engineering | Can build accessible pages and DOM interactions |
| 3 | React + TypeScript | Can build a structured SPA with routing and forms |
| 4 | HTTP + REST + Web APIs | Can reason about frontend–backend traffic |
| 5 | Java or Python backend | Can implement a secure REST API |
| 6 | Databases | Can design and query SQL independently |
| 7 | Architecture | Can choose structure for a reason |
| 8 | Testing, security, performance | Can protect, verify, and measure software |
| 9 | Git, Docker, CI/CD, cloud | Can deliver a running system |
| 10 | Agile + professional engineering | Can work as a team engineer |

> The reference material is the knowledge map. Deshmukh Technologies projects are the proof. A trainee completes this program when they can independently build, test, debug, explain, document, and maintain a full-stack application.

DTTP — DESHMUKH TECHNOLOGIES TRAINEE PROGRAM

# Self-Directed Full-Stack Developer

Ten levels. One practice stack. Real projects.

Learn. Build. Solve. Review. Deploy. Grow.